

Parallel Algorithms: Heat Equation

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

2D Heat Equation PDE

$$\underbrace{\rho c \frac{\partial T}{\partial t}}_{\text{change in temperature over time}} = \underbrace{k \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)}_{\text{heat flow along both directions}} = k \nabla^2 T$$

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

- density ρ , heat capacity c , and thermal conductivity k
- T is the temperature measured at point (x, y) and time t
- Laplace operator (Laplacian):

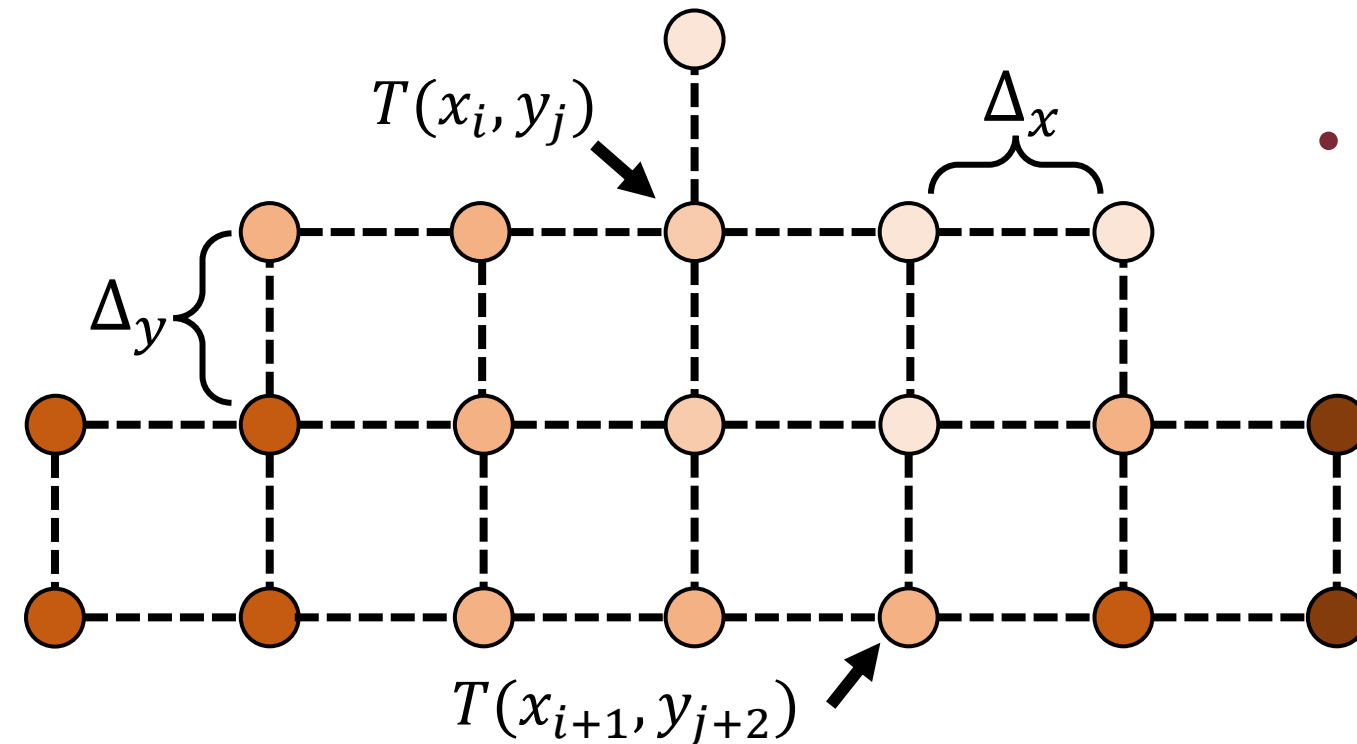
$$\nabla^2 = \sum \frac{\partial^2}{\partial x_i^2}$$

- Diffusion equation where $\alpha = \frac{k}{\rho c}$ is the material's thermal diffusivity

Discretization

- Track the discrete temperature on a grid:

$$T_{i,j}^n = T[i, j, n] = T(x_i, y_j, t_n)$$



- Each discrete spatial location x_i, y_i stores the temperature at time t_n
- Spacing between grid points:

$$\Delta_x = x_{i+1} - x_i$$

- We update this grid to calculate the temperature for each time step:

$$\Delta_t = t_{n+1} - t_n$$

- Step values are specified by the user
- Related to actual spatial locations and time points:

$$x_i = x_0 + i \times \Delta_x$$

Finite Difference Calculus

- Finite difference methods:

$$\frac{\partial f}{\partial x} \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2\Delta_x} \quad O(\Delta_x^2)$$

$$\approx \frac{f(x_{i+1}) - f(x_i)}{\Delta_x} \quad O(\Delta_x)$$

$$\approx \frac{f(x_i) - f(x_{i-1}))}{\Delta_x} \quad O(\Delta_x)$$

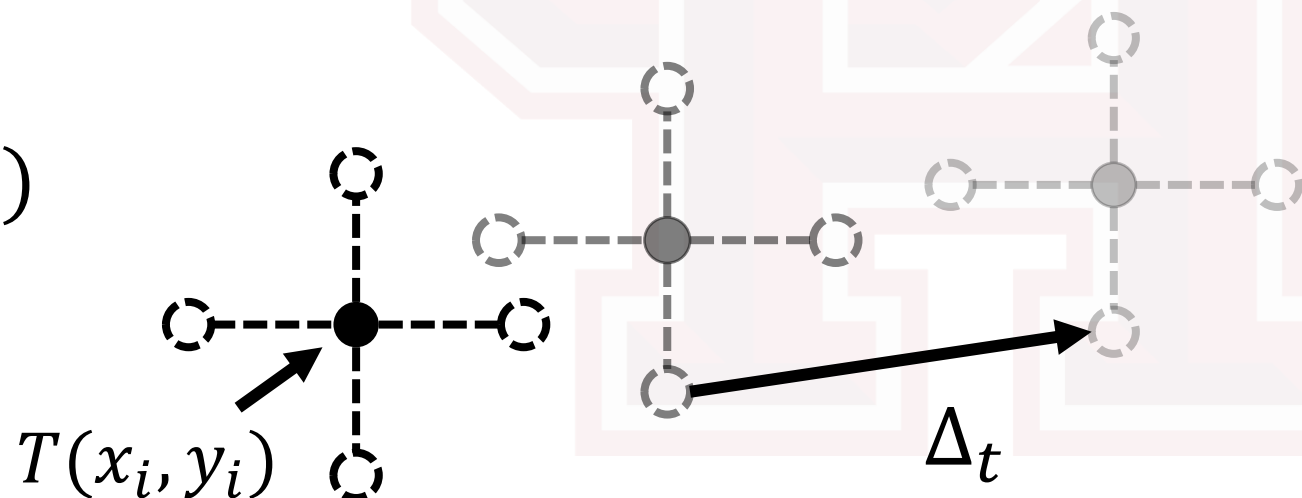
- Advantages/disadvantages of each form?

- We know all spatial locations:

$$\frac{\partial T}{\partial x} \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2\Delta_x} \quad \frac{\partial T}{\partial y} \approx \frac{f(y_{j+1}) - f(y_{j-1}))}{2\Delta_y}$$

- We don't know the future (t_{n+1}):

$$\frac{\partial T}{\partial t} \approx \frac{f(x_n) - f(x_{n-1}))}{\Delta_t}$$



Explicit Method

- Second-derivative finite difference:

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{\Delta_x^2}$$

- Given the temperature at any time t_n , approximate the temperature at t_{n+1} :

$$\frac{\partial T}{\partial t} = \frac{T^{n+1} - T^n}{\Delta_t}$$
$$T^{n+1} = T^n + \Delta_t \boxed{\frac{\partial T}{\partial t}}$$

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

- Setting $h = \Delta_x = \Delta_y$ simplifies the math:

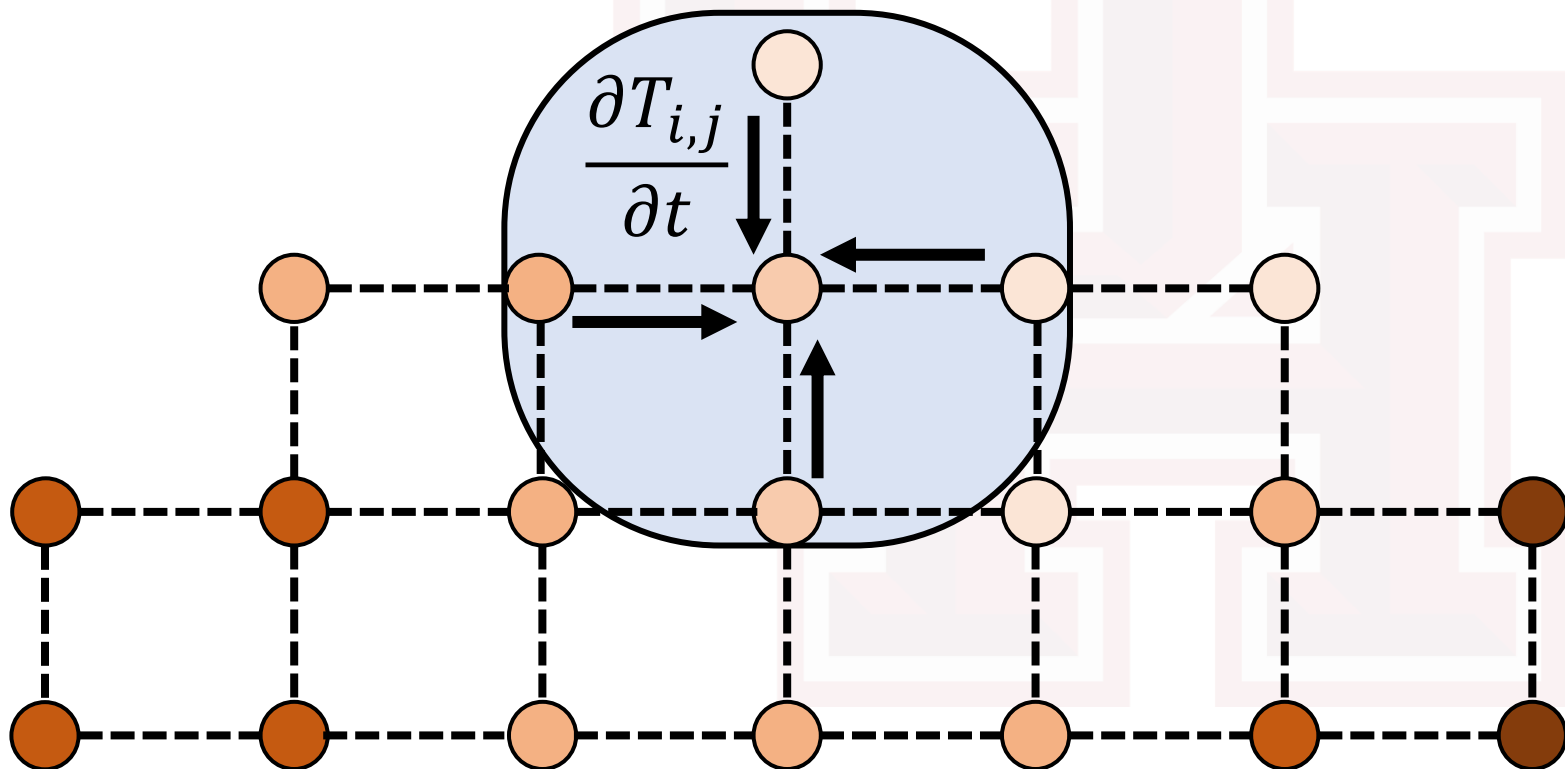
$$\frac{\partial T}{\partial t} \approx \frac{\alpha}{h^2} (T_{i+1,j}^n + T_{i-1,j}^n + T_{i,j+1}^n + T_{i,j-1}^n - 4T_{i,j}^n)$$

- This is known as an explicit method (Euler's method) for calculating T^{n+1}

Algorithm

$$\frac{\partial T_{i,j}^n}{\partial t} \approx \frac{\alpha}{h^2} (T_{i+1,j}^n + T_{i-1,j}^n + T_{i,j+1}^n + T_{i,j-1}^n - 4T_{i,j}^n)$$

$$T_{i,j}^{n+1} = T_{i,j}^n + \Delta_t \frac{\partial T_{i,j}^n}{\partial t}$$



Simulation Error

Approximation

- Finite difference methods are based on Taylor series:

$$f(x) = f(c) + f'(c)(x - c) + E$$

- where $c = x_i$ and $x = x_{i+1}$:

$$f(x_{i+1}) = f(x_i) + f'(x_i)\Delta_x + E$$

- solve for the derivative:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{\Delta_x} + E$$

- Second-order second derivative:

$$\begin{aligned} f''(x_i) &= \frac{\frac{f(x_{i+1}) - f(x_i)}{\Delta_x} - \frac{f(x_i) - f(x_{i-1}))}{\Delta_x}}{\Delta_x} + E \\ &= \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{\Delta_x^2} + E \end{aligned}$$

Error and Stability

- Error term depends on sample spacing: $E \in O(\Delta_x^2)$
- Smaller samples provide better accuracy but increase calculations
- Stability is based on the Courant-Friedrichs-Lewy (CFL) condition:

$$\alpha \Delta_t \left(\frac{1}{\Delta_x^2} + \frac{1}{\Delta_y^2} \right) \leq \frac{1}{2}$$

- Select Δ_x , Δ_y , and Δ_t to optimize accuracy, stability, and efficiency

Implementation 1: Follow the Math

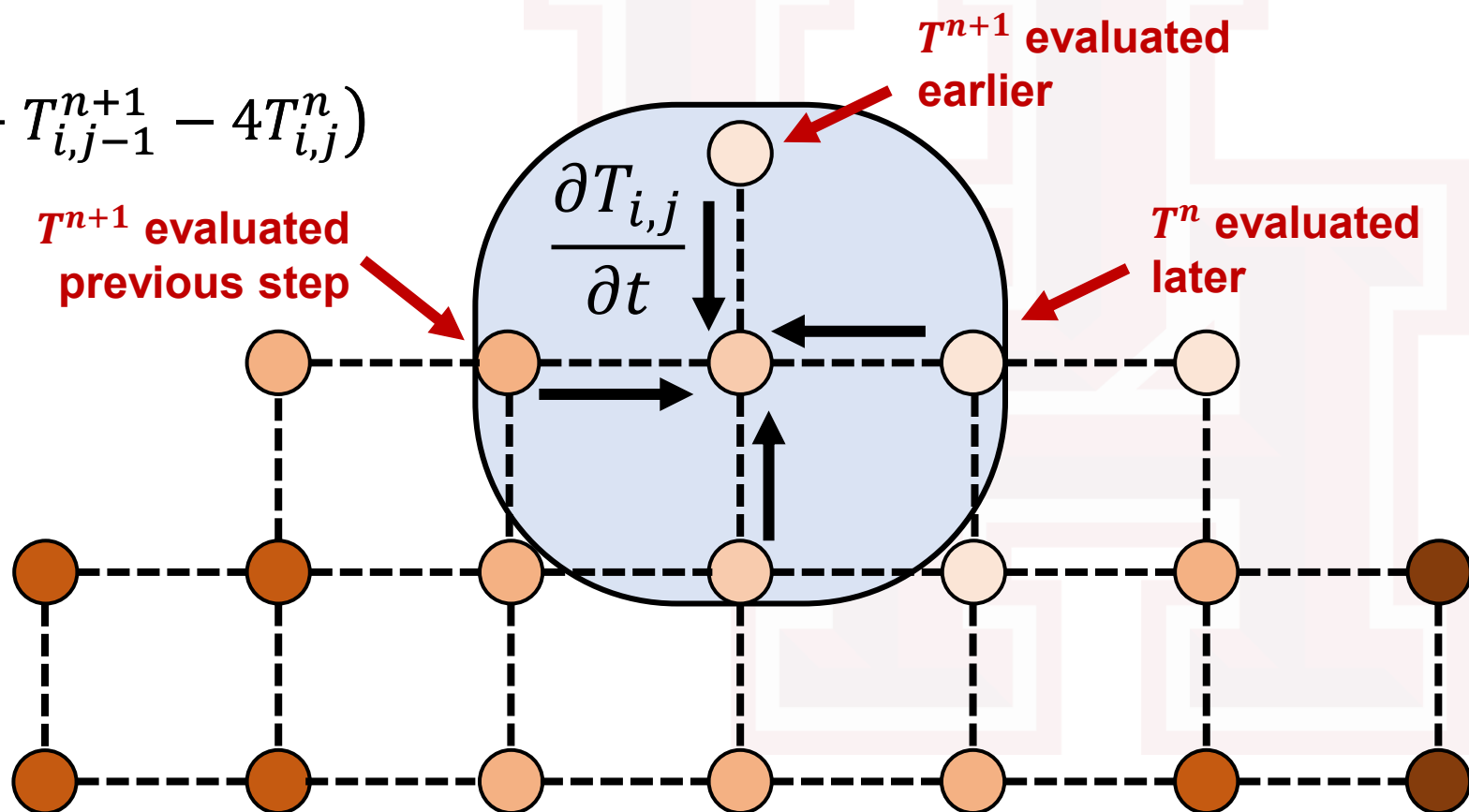
```
void heat(float* A, float alpha, float h, float dt, int S, float epsilon) {  
    float* B = new float[S * S];    2X memory!    // create a new array to store the output  
    float c = alpha / (h * h);        // pre-compute the spatial diffusion term  
    while(true) {                    // loop indefinitely  
        float diff = 0.0f;           // track how much the simulation result has changed  
        for(int j=1; j<S; j++) {     // iterate through each pixel (why y then x?), note boundaries  
            for(int i=1; i<S; i++) {  
                // calculate the partial derivative with respect to time (main calculation)  
                dT_dt = c * (A[j*S+i+1] + A[j*S+i-1] + A[(j+1)*S+i] + A[(j-1)*S+i] - 4*A[j*S+i];  
                B[j*S+i] = A[j*S+i] + dt * dT_dt;    extra caching    // use explicit integration  
                diff += fabs(B[j*S+i] - A[j*S+i]);    // sum the absolute change  
            }  
        }  
        float* tmp = A;  A = B;  B = tmp;           // swap pointers  
        if(diff / (S*S) < epsilon) { delete B; return; } // delete the temporary array and return  
    }  
}
```

Implementation 2: Single-Process Efficiency

$$\frac{\partial T_{i,j}^n}{\partial t} \approx \frac{\alpha}{h^2} (T_{i+1,j}^n + T_{i-1,j}^n + T_{i,j+1}^n + T_{i,j-1}^n - 4T_{i,j}^n)$$

$$\frac{\partial T_{i,j}^n}{\partial t} \approx \frac{\alpha}{h^2} (T_{i+1,j}^n + T_{i-1,j}^{n+1} + T_{i,j+1}^n + T_{i,j-1}^{n+1} - 4T_{i,j}^n)$$

$$T_{i,j}^{n+1} = T_{i,j}^n + \Delta t \frac{\partial T_{i,j}^n}{\partial t}$$



Implementation 2: Single-Process Efficiency

```
void heat(float* A, float alpha, float h, float dt, int S, float epsilon) {
    float c = alpha / (h * h);           // pre-compute the spatial diffusion term
    while(true) {                         // loop indefinitely
        float diff = 0.0f;               // track how much the simulation result has changed
        for(int j=1; j<S-1; j++) {       // iterate through each pixel (why y then x?), note boundaries
            for(int i=1; i<S-1; i++) {
                // calculate the partial derivative with respect to time (main calculation)
                float prev = A[j*S+i];
                dT_dt = c * (A[j*S+i+1] + A[j*S+i-1] + A[(j+1)*S+i] + A[(j-1)*S+i] - 4*prev; // dependencies
                A[j*S+i] += dt * dT_dt;           // use explicit integration
                diff += fabs(A[j*S+i] - prev);     // sum the absolute change // dependencies
            }
        }
        if(diff / (S*S) < epsilon) return; // delete the temporary array and return
    }
}
```

Dependencies and Conflicting Requirements

Implementation 1 (write to temp array)

- Each grid point is independent within one time step
- Requires twice as much memory
- Requires larger cache (or more misses)
- Memory and cache size is one of the first sacrifices in parallel processors

Implementation 2 (single array)

- No additional memory
- Better cache coherence
- Both cases have a dependency in the terminating condition:

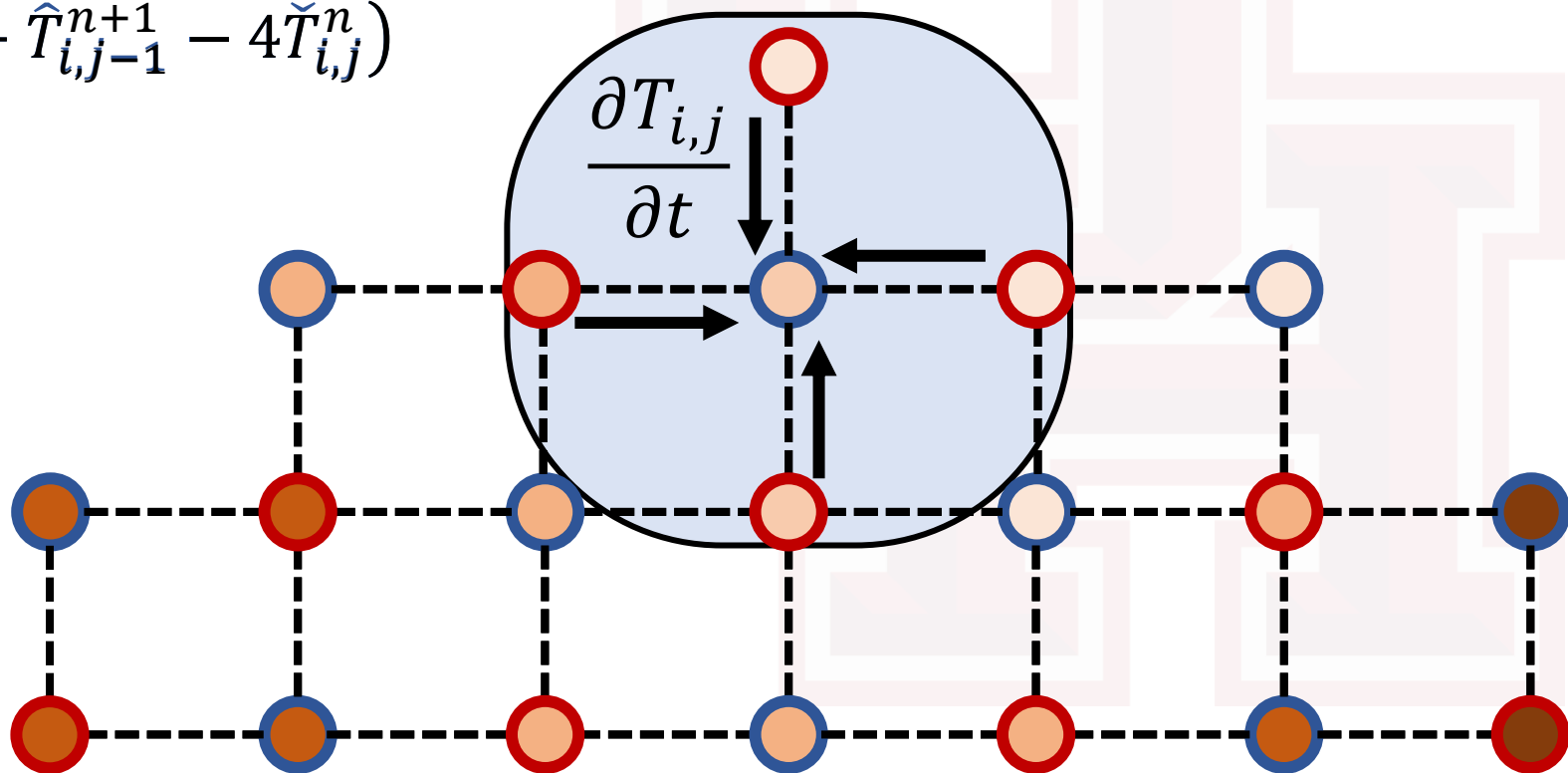
```
// .....  
    diff += fabs(B[j*S+i] - prev);  
}  
}  
if(diff / (S*S) < epsilon) return;
```

Implementation 3: Alternating Grid

$$\frac{\partial \hat{T}_{i,j}^n}{\partial t} \approx \frac{\alpha}{h^2} (\check{T}_{i+1,j}^n + \check{T}_{i-1,j}^n + \check{T}_{i,j+1}^n + \check{T}_{i,j-1}^n - 4\hat{T}_{i,j}^n)$$

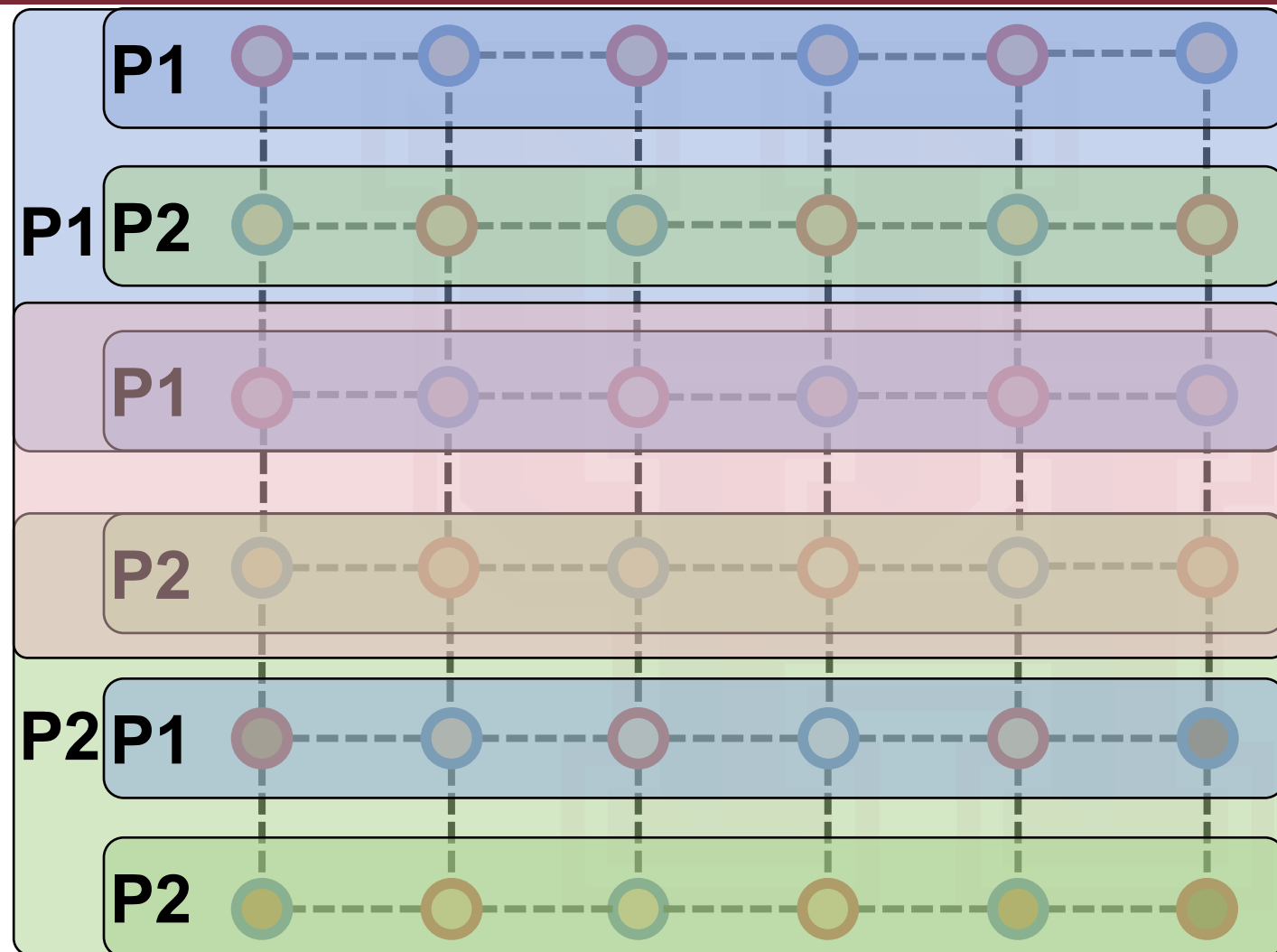
$$\frac{\partial \check{T}_{i,j}^n}{\partial t} \approx \frac{\alpha}{h^2} (\hat{T}_{i+1,j}^n + \hat{T}_{i-1,j}^{n+1} + \hat{T}_{i,j+1}^n + \hat{T}_{i,j-1}^{n+1} - 4\check{T}_{i,j}^n)$$

$$T_{i,j}^{n+1} = T_{i,j}^n + \Delta_t \frac{\partial T_{i,j}^n}{\partial t}$$



Implementation 3: Parallelization

- How can we evaluate $\frac{\partial T}{\partial t}$ in parallel?
- Red vs. Blue – will this work?
 - No, red and blue are dependent on each other
 - Calculate all red, then all blue, etc.
- Interleaving
- Blocking
 - Less communication required
 - Distributed systems (send blocks to different computers)
- Depends on architecture and problem (computation vs. memory access)



Implementation 3: Parallel Red/Blue (w/ Race Condition)

```
void heat_red(float* A, float alpha, float h, float dt, int S, float epsilon, int thread_min, int thread_max) {
    float c = alpha / (h * h);
    while(true) {
        float diff = 0.0f;
        for(int j=thread_min; j<thread_max; j++) {
            for(int i=1; i<S-1; i++) {
                // calculate the partial derivative with respect to time (main calculation)
                float prev = A[j*S+i];
                dT_dt = c * (A[j*S+i+1] + A[j*S+i-1] + A[(j+1)*S+i] + A[(j-1)*S+i] - 4*prev;
                A[j*S+i] += dt * dT_dt;
                diff += fabs(A[j*S+i] - prev);
            }
        }
        if(diff / (S*S) < epsilon) return;
    }
}
```

// pre-compute the spatial diffusion term
// loop indefinitely
// track how much the simulation result has changed
// iterate across all lines associated with this thread
// use explicit integration
// sum the absolute change **dependency (race condition)**
// delete the temporary array and return

Implementation 3: Parallel Red/Blue (w/ Race Condition)

```
void heat_red(float* A, float alpha, float h, float dt, int S, float epsilon, int thread_min, int thread_max) {
    float c = alpha / (h * h);
    while(true) {
        float diff = 0.0f;
        for(int j=thread_min; j<thread_max; j++) {
            for(int i=1; i<S-1; i++) {
                // calculate the partial derivative with respect to time (main calculation)
                float prev = A[j*S+i];
                dT_dt = c * (A[j*S+i+1] + A[j*S+i-1] + A[(j+1)*S+i] + A[(j-1)*S+i] - 4*prev;
                A[j*S+i] = A[j*S+i] + dt * dT_dt;
                // use explicit integration
                float new_val = A[j*S+i] + dt * dT_dt;
                // apply new value to the array (check the variable before writing)
                A[j*S+i] = new_val;
            }
            diff += dT_dt;
        }
        if(diff / (S*S) < epsilon) return;
    }
}
```

STILL BAD – now you're forcing threads to stall and wait for others to write

Implementation 3: Parallel Red/Blue

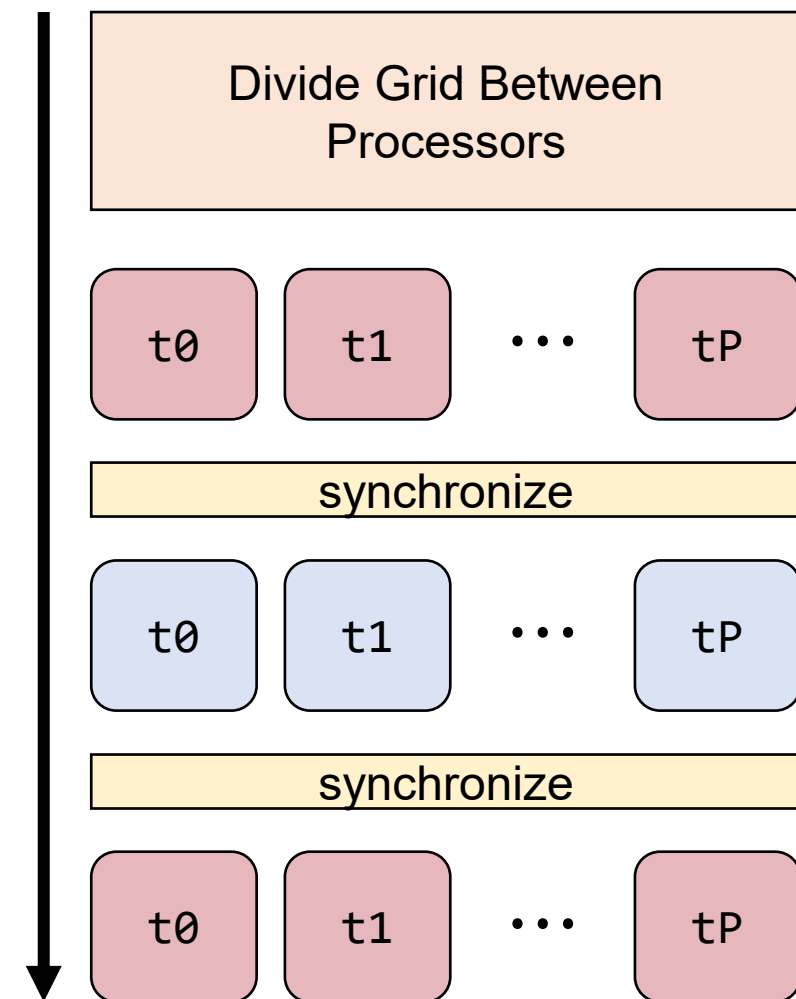
```
float diff = 0.0f;
void heat_red(float* A, float alpha, float h, float dt, int S, float epsilon, int thread_min, int thread_max) {
    float c = alpha / (h * h);
    while(true) {
        float thread_diff = 0.0f;
        for(int j=thread_min; j<thread_max; j++) {
            for(int i=1; i<S-1; i++) {
                float prev = A[j*S+i];
                dT_dt = c * (A[j*S+i+1] + A[j*S+i-1] + A[(j+1)*S+i] + A[(j-1)*S+i] - 4*prev;
                A[j*S+i] = A[j*S+i] + dt * dT_dt;
                thread_diff += fabs(A[j*S+i] - prev);
            }
        }
        atomicAdd(diff, thread_diff);
        if(diff / (S*S) < epsilon) return;
    }
}
```

// global value storing the total change
// pre-compute the spatial diffusion term
// loop indefinitely
// stores change across THIS thread
// iterate across all lines associated with this thread

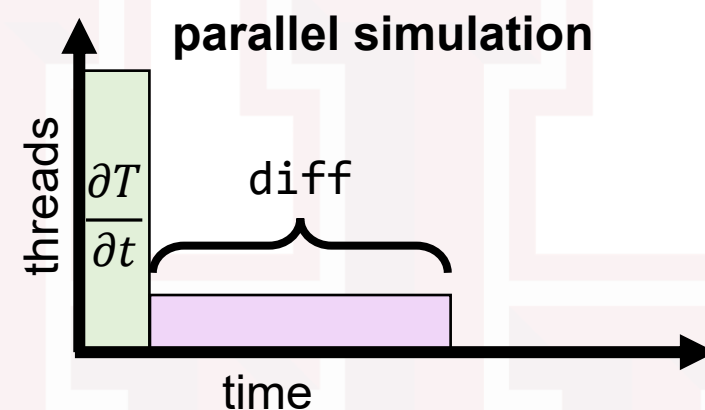
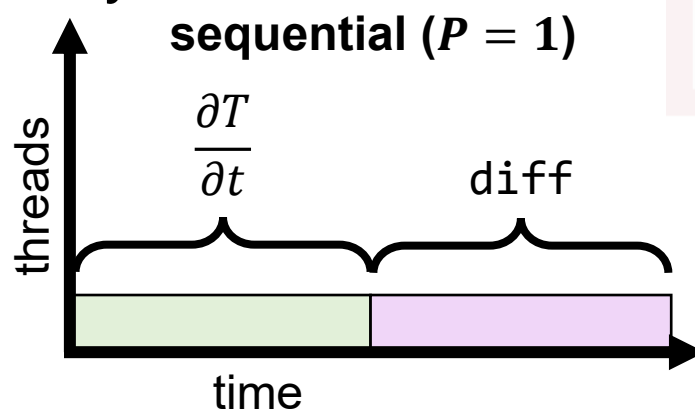
// use explicit integration
// sum the absolute change across THIS thread

// add the global difference atomically
// delete the temporary array and return

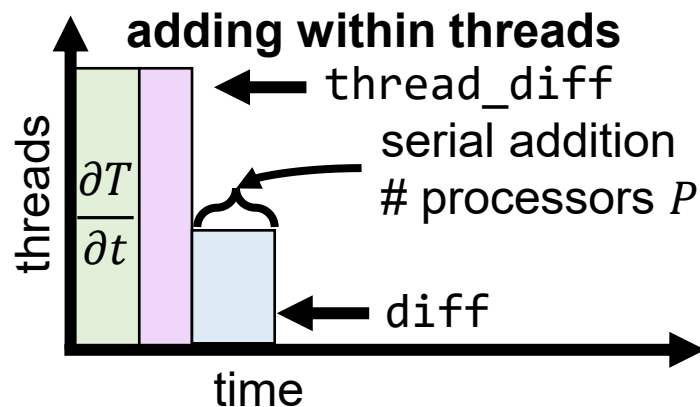
Implementation



- Synchronization limits parallelism



- Summing per-thread (thread_diff) with atomic operations (atomicAdd):



What happens when the number of processors/threads approaches the number of grid points ($P \rightarrow S^2$)?

Amdahl's Law

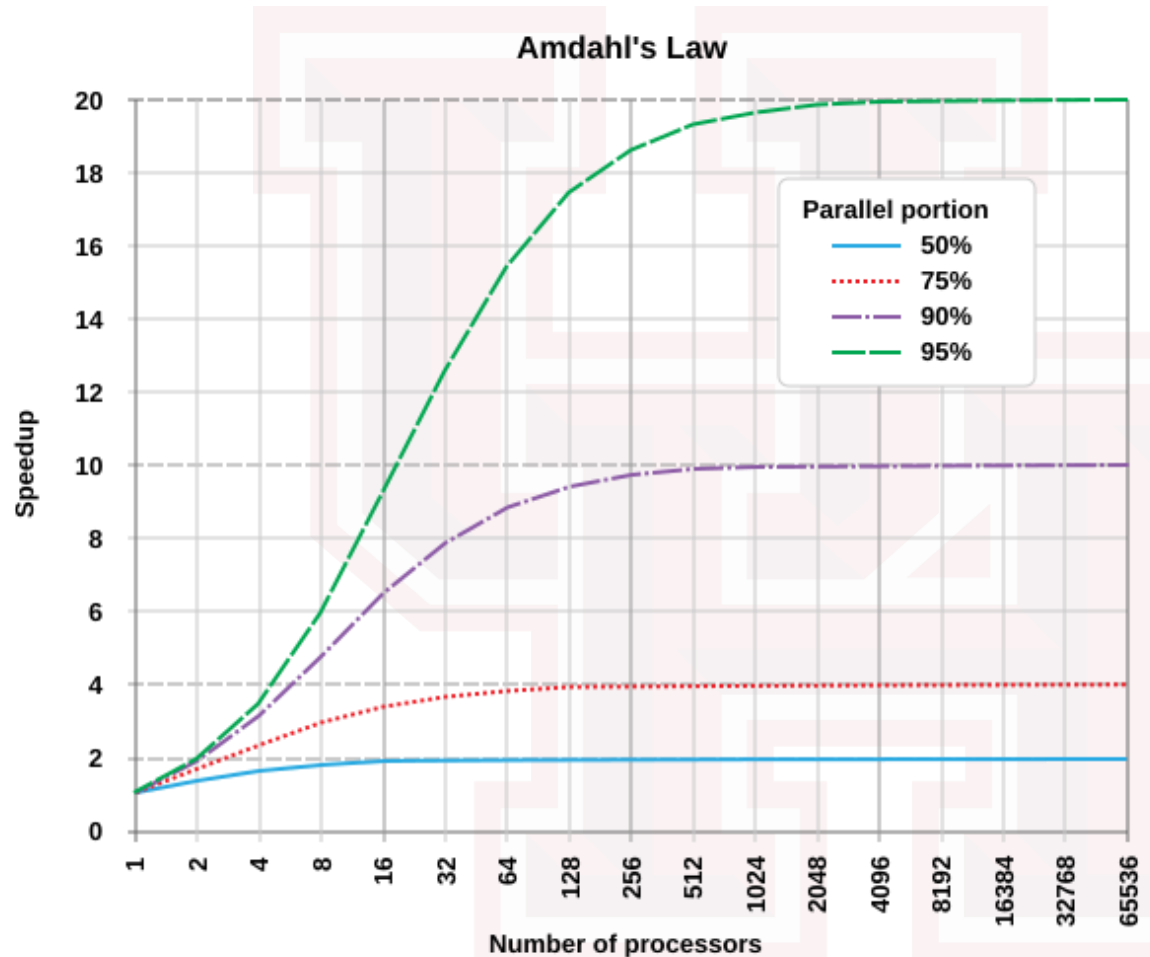
- Your ability to parallelize a program is limited by the serial parts:

$$S = \frac{1}{(1 - p) + \frac{p}{n}}$$

- S is the speedup
- p is the parallel portion of the program
- n is the number of processors

- What is your expected speedup with 64 processors if 99.9% of your program can be parallelized?

$$\frac{1}{0.001 + \frac{0.999}{64}} \approx 60X$$



Gustafson's Law

- Alternative approach gives more optimistic results:
$$S = 1 - p + pn$$
 - S is the speedup
 - p is the parallel portion of the program
 - n is the number of processors
- What is your expected speedup with 64 processors if 99.9% of your program can be parallelized?
$$.001 + .999 \times 64 \approx 64X$$

- Why different answers (60 vs. 64)?
- Ahmdal's law represents a fixed problem size
- Gustafson's law assumes the parallel part will grow to **fully occupy the available processors**
- How would this apply to our heat equation example?